

Chat285

Design & Architecture Document

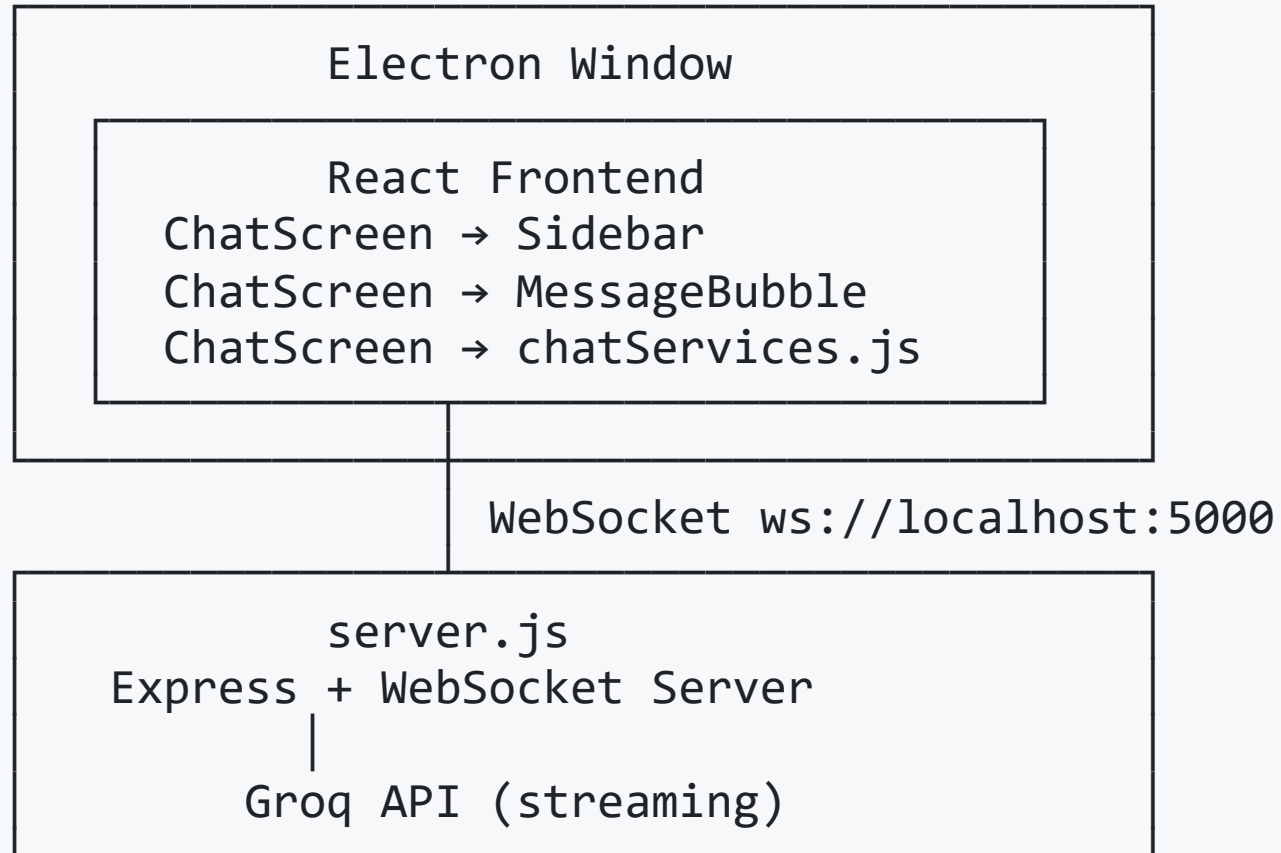
Tech Stack

Layer	Technology
Frontend	React 19, Vite 7
Rendering	react-markdown, remark-math, rehype-katex
Backend	Node.js, Express 5, ws
AI	Groq API (OpenAI-compatible)
Desktop	Electron
Testing	Vitest + Testing Library
Storage	localStorage

Project Structure

```
src/  
├── lib/  
│   ├── models/           message.js, conversation.js  
│   ├── screens/          ChatScreen.jsx  
│   ├── services/         chatServices.js  
│   └── widgets/          MessageBubble.jsx, Sidebar.jsx  
├── server.js  
├── main.jsx  
└── styles.css  
electron/  
├── main.cjs  
└── preload.cjs  
tests/  
├── unit_tests/  
├── integration_tests/  
├── acceptance_tests/  
├── regression_tests/  
└── lib/testUtils.js
```

System Architecture



Message Flow

1. User sends message → `sendMessage(input)`
2. User message added to state + localStorage
3. `sendMessageToServer()` sends `{ message, history }` over WebSocket
4. Server prepends system prompt, calls Groq with `stream: true`
5. Each token sent to client as `{ type: 'token', token }`
6. `onToken()` appends token to streaming bubble
7. Server sends `{ type: 'done' }`
8. `streamingID` cleared → bubble switches to ReactMarkdown

WebSocket Protocol

Direction	Type	Payload
Client → Server	—	{ message, history }
Server → Client	token	{ type, token }
Server → Client	done	{ type }
Server → Client	error	{ type, message }

Data Models

Message

```
{ messageId, role, content, timestamp }
```

Conversation

```
{ conversationID, title, createdAt, messages[] }
```

localStorage keys

- `conversations` — all conversation objects
- `activeConversationID` — currently selected UUID

Key Design Decisions

- **localStorage over sessionStorage** — conversations survive restarts
- **Client owns history** — server is stateless, restarts don't lose context
- **WebSocket over HTTP** — enables real-time token streaming
- **Ref + State for streamingID** — ref avoids stale closures, state triggers re-renders
- **Electron CommonJS (.cjs)** — avoids conflict with `"type": "module"` in `package.json`

Testing Strategy

Layer	Purpose
Unit	Individual components in isolation
Integration	Multiple components working together
Acceptance	Complete user stories end to end
Regression	Verify fixed bugs don't return
lib/testUtils.js	Shared factories and setup helpers

Component Responsibilities

Component	Responsibility
<code>ChatScreen.jsx</code>	All app state, WebSocket lifecycle, conversation management
<code>Sidebar.jsx</code>	Purely presentational, no state
<code>MessageBubble.jsx</code>	Plain text while streaming, ReactMarkdown when done
<code>chatServices.js</code>	WebSocket connect, send, and token callback
<code>server.js</code>	Receive message, stream Groq response token by token